

The use of copyrighted materials in all formats, including the creation, online delivery, and use of digital copies of copyrighted materials, must be in compliance with U.S. Copyright Law (<http://www.copyright.gov/title17/>). Materials may not be reproduced in any form without permission from the publisher, except as permitted under U.S. copyright law. **Copyrighted works are provided under Fair Use Guidelines only to serve personal study, scholarship, research, or teaching needs.**

Maurizio Gabbrielli and Simone Martini

Programming Languages: Principles and Paradigms

 Springer

Prof. Dr. Maurizio Gabbrielli
Università di Bologna
Bologna
Italy

Prof. Dr. Simone Martini
Università di Bologna
Bologna
Italy

Series editor
Ian Mackie

Advisory board

Samson Abramsky, University of Oxford, UK
Chris Hankin, Imperial College London, UK
Dexter Kozen, Cornell University, USA
Andrew Pitts, University of Cambridge, UK
Hanne Riis Nielson, Technical University of Denmark, Denmark
Steven Skiena, Stony Brook University, USA
Iain Stewart, University of Durham, UK
David Zhang, The Hong Kong Polytechnic University, Hong Kong

ISSN 1863-7310

ISBN 978-1-84882-913-8

e-ISBN 978-1-84882-914-5

DOI 10.1007/978-1-84882-914-5

Springer London Dordrecht Heidelberg New York

British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

Library of Congress Control Number: 2010922995

Translated from the original Italian edition: "Linguaggi di programmazione: principi e paradigmi", published in 2006 by McGraw-Hill Companies, Publishing Group Italia.

© Springer-Verlag London Limited 2010

Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Designs and Patents Act 1988, this publication may only be reproduced, stored or transmitted, in any form or by any means, with the prior permission in writing of the publishers, or in the case of reprographic reproduction in accordance with the terms of licenses issued by the Copyright Licensing Agency. Enquiries concerning reproduction outside those terms should be sent to the publishers.

The use of registered names, trademarks, etc., in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant laws and regulations and therefore free for general use.

The publisher makes no representation, express or implied, with regard to the accuracy of the information contained in this book and cannot accept any legal responsibility or liability for any errors or omissions that may be made.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

Chapter 12

The Logic Programming Paradigm

In this chapter we analyse the other paradigm which, together with functional programming, supports declarative programming. The logic programming paradigm includes both theoretical and fully implemented languages, of which the best known is surely PROLOG. Even if there are big differences of a pragmatic and, for some, of a theoretical nature between these languages, they all share the idea of interpreting computation as logical deduction.

In this chapter, we will therefore examine these concepts while trying to limit the theoretical part. We also adopt the approach that has characterised the rest of the text while examining this paradigm. We do not mean therefore to teach programming in PROLOG, even if we present various examples of real programs, but we do intend to provide enough basis for understanding and, in a short time, mastering this and other logic programming languages.

12.1 Deduction as Computation

A well-known slogan due to R. Kowalski, exactly captures the concepts that underpin the activity of programming: $\text{Algorithm} = \text{Logic} + \text{Control}$. According to this “equation”, the specification of an algorithm, and therefore its formulation in programming languages, can be separated into three parts. On the one side, the logic of the solution is specified. That is, the “what” must be done is defined. On the other, those aspects related to control are specified, and therefore the “how” of finding the desired solution is clarified. The programmer who uses a traditional imperative language must take account of both these components using the mechanisms which we have variously analysed in the chapters that precede this one. Logic programming, on the other hand, was originally defined with the idea of cleanly separating these two aspects. The programmer is only required, at least in principle, to provide a logical specification. Everything related to control is relegated to the abstract machine. Using a computational mechanism based on a particular deduction rule (resolution), the interpreter searches through the space of possible solutions for the one specified

by the “logic”, defining in this way the sequence of operations necessary to reach the final result.

The basis for this view of computation as logical deduction can be traced back to the work of K. Gödel and J. Herbrand in the 1930s. In particular, Herbrand anticipated, even in an incompletely formal way, some ideas on the process of unification, which, as we will see, forms the basic computational mechanism of logic programming languages.

It was not until the 1960s that a formal definition (due to A. Robinson) of this process appeared and only ten years later (in the early 1970s) it was realised that formal automatic deduction of a particular kind could be interpreted as a computational mechanism. The first programming languages in the logic programming paradigm were created, among which PROLOG (the name is an acronym for PROgramming in LOGic, for more information on the story of this language, see Sect. 13.4).

Today there are many implemented versions of PROLOG and there exist various other languages in this paradigm (as far as applications are concerned, those including constraints are of particular interest). All of these languages (and PROLOG was the first) allow the use of constructs permitting the specification of control for reasons of efficiency. Since these constructs do not have a direct logical interpretation, they make the semantics of the language more complicated, and cause the loss of part of the purely declarative nature of the logic paradigm. This notwithstanding, we are still dealing with logic programming languages, even including these “impure” aspects, which require the programmer to do little more than formulate (or declare) the specification of the problem to be solved. In some cases, the resulting programs are really surprising in their brevity, simplicity and clarity, as we will see in the next section.

Terminological Note Logic programming, which constitutes the theoretical formalism, is distinguished from PROLOG, a language which has different implementations and for which a standard has recently been defined. The concepts we introduce below, if not otherwise stated, are valid for both formalisms. The important differences will be explicitly pointed out. The programming examples which we include (usually in this font) are all PROLOG code which could be run on some implementation of the language. The theoretical concepts, on the other hand, even when they are valid for PROLOG, use mathematical concepts and are rendered in *italic characters*. Moreover, we will follow the PROLOG convention that we always write variables as strings of characters beginning with an upper-case letter. Finally note that, as mentioned before, there exist several other languages, different from PROLOG, in the logic programming paradigm. We call these logic programming languages, or logic languages, for short.

We try to substantiate what we said in the last section with an example which will be fairly informal, at least for now, given that we have introduced neither the syntax nor the semantics of logic languages.

Reasoning in a declarative fashion, on the other hand, we can proceed as follows. First, we need a list which we will call `Ls`. This list will have to contain 27 elements, something that we can specify using a unary predicate (that is, a relation symbol)¹ `list_of_27`. If we write `list_of_27(Ls)`, we mean, therefore, to say that `Ls` must be a list of 27 elements. In other words, `list_of_27` defines a relation formed of all possible lists of 27 elements. To achieve this aim, we can define `list_of_27` as follows:

where the part to the left of the $:-$ symbol denotes what is being defined, while the part to the right of $:-$ indicates the definition. The $=$ denotes an equality concept whose definition is left to intuition, for the present (it is different, however, from assignment). Anticipating the PROLOG notation for lists, here we write $[X1, X2, \dots, Xn]$ to denote the list that contains the n variables $X1, X2, \dots, Xn$ (see Sect. 12.4.5 for more details about lists).

In order to satisfy our specification, the list, L_S , in addition to being composed of 27 elements, will have to contain a sublist² in which the number 1 appears followed by any other number, by another occurrence of the number 1, then another number and finally a last occurrence of the number 1. Such a sublist can be specified as

¹ Recall that unary means that it has a single argument while n -ary means that there are n arguments.

²Recall that L_i is a sublist of L_S if L_S is obtained by concatenating a (possibly empty) list with L_i and with another list (itself also possibly empty).

```
[1,X,1,Y,1]
```

where X and Y are variables. More efficiently, we can write:

```
[1,_,1,_,1]
```

where $_$ is the *anonymous* variable, that is a variable whose name is of no interest and which is distinct from all other variables present, including all other anonymous variables. Assuming that we have available a binary predicate `sublist(X,Y)` whose meaning is that the first argument (X) is a sublist of the second (Y),³ our requirement therefore can be expressed by writing:

```
sublist([1,_,1,_,1], Ls)
```

Moving on to number 2 and reasoning in a similar fashion, we obtain that the list Ls must also contain the sublist:

```
[2,_,_,2,_,_,2]
```

and therefore the following must also be true:

```
sublist([2,_,_,2,_,_,2], Ls)
```

We can repeat this reasoning as far as number 9, hence the `sol` program that we want to produce can be described as follows:

```
sol(Ls) :-                                     1
    list_of_27(Ls),
    sublist([9,_,_,_,_,_,_,_,_,9,_,_,_,_,_,_,_,_,_,9], Ls),      3
    sublist([8,_,_,_,_,_,_,_,_,8,_,_,_,_,_,_,_,_,_,8], Ls),
    sublist([7,_,_,_,_,_,_,_,_,7,_,_,_,_,_,_,_,_,_,7], Ls),      5
    sublist([6,_,_,_,_,_,_,_,_,6,_,_,_,_,_,_,_,_,_,6], Ls),
    sublist([5,_,_,_,_,_,_,_,_,5,_,_,_,_,_,_,_,_,_,5], Ls),      7
    sublist([4,_,_,_,_,_,_,_,_,4,_,_,_,_,_,_,_,_,_,4], Ls),
    sublist([3,_,_,_,_,_,_,_,_,3,_,_,_,_,_,_,_,_,_,3], Ls),      9
    sublist([2,_,_,_,_,_,_,_,_,2,_,_,_,_,_,_,_,_,_,2], Ls),
    sublist([1,_,_,_,_,_,_,_,_,1,_,_,_,_,_,_,_,_,_,1], Ls).      11

list_of_27(Ls) :-                               13
Ls = [_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_].
```

where, as before, the part to the right of the `:-` symbol is the definition of what lies to its left. What we have written above says that to find a solution to our problem (`sol(Ls)` in line 1), all the properties described in lines 2–11 must be satisfied. The commas that separate the various predicates are to be interpreted as “and”, that is as conjunctions in the logical sense. What we have provided therefore is nothing

³The definition of `sublist` is given in Sect. 12.4.5.

more than a specification that, in substance, formally repeats the formulation of the problem and which, as we will see below, can be interpreted in purely logical terms.

This declarative reading, for all of its elegance and compactness, would not be of great use as the solution to our problem if it were not accompanied by a procedural interpretation and must therefore be translated into a conventional programming language. The important characteristics of the paradigm that we are considering is precisely that the (logical) specifications we write are to all intents and purposes executable programs. The code described above in lines 1–14 (plus the definition of `sublist` in Sect. 12.4.5) is indeed a genuine PROLOG program that can be evaluated by an interpreter to obtain the desired solution.

In other words, our specification can also be read in a procedural fashion, as follows. Line (1) contains the declaration of a procedure called `sol`; it has a single formal parameter, `Ls`. The body of this procedure is defined by lines (2) to (11), where we find the following ten procedure calls. On line (2), procedure `list_of_27` is called with actual parameter `Ls` which is defined as we have seen above and which therefore instantiates⁴ its actual parameter into a list of 27 anonymous variables. Therefore, on line (3), we have the call:

```
sublist([9,_,_,_,_,_,_,_,_,_,9,_,_,_,_,_,_,_,_,_,_,9], Ls)
```

We assume that this call arranges matters so that the list which appears as the first parameter is a sublist of the list which is bound to the second parameter (`Ls`), possibly by instantiating the variable `Ls`. Similarly for the other calls until line (11). Parameter passing occurs in a way similar to call by name and, in PROLOG, the order in which the different procedure calls appear in the text specifies the order of evaluation (in the case of pure logic programs, on the other hand, no order is specified).

Given the preceding definition of procedure `sol`, the call `sol(Ls)` returns `Ls` instantiated with a solution to the problem, such as for example the following:

```
1,9,1,2,1,8,2,4,6,2,7,9,4,5,8,6,3,4,7,5,3,9,6,8,3,5,7.
```

Successive calls to the same procedure will allow us also to obtain the other solutions to the problem.

With this procedural interpretation, one has therefore a true programming language which allows us to express, in a compact and relatively simple fashion, programs that solve even very complex problems. For this power, the language pays the penalty of efficiency. In the preceding program, despite its apparent simplicity, the computation performed by the language's abstract machine is very complex, given that the interpreter must try the various combinations of possible sublists until it finds the one that satisfies all the conditions. In these search processes, a *backtracking* mechanism is used. When the computation arrives at a point at which it cannot

⁴Even if the terminology is intuitively clear: by instantiation of a variable is meant substituting for it a syntactic object. We will give a precise definition in Sect. 12.3.

proceed, the computation that has been performed is undone so that a decision point can be reached, if it exists, at which an alternative is chosen that is different from the previous one (if this alternative does not exist, the computation terminates in failure). It is not difficult to see that, in general, this search process can have exponential complexity.

12.2 Syntax

Logic programs are sets of logic formulæ of a particular form. We begin therefore with some basic notions for defining the syntax.

The logic of interest here is first-order logic; it is also called *predicate calculus*. The terminology refers to the fact that symbols are used to express (or, in a more old-fashioned terminology, to “predicate”) properties of elements of a fixed domain of discourse, $\mathcal{D}$. More expressive logics (second, third, etc., order) also permit predicates whose arguments are more complicated objects such as sets and functions over $\mathcal{D}$ (second order), sets of functions (third order), etc., in addition to elements of $\mathcal{D}$.

12.2.1 The Language of First-Order Logic

If we are to speak of predicate calculus (and therefore of logic programs), we have to define the language. *The language of first-order logic* consists of three components:

1. An *alphabet*.
2. *Terms* defined over this alphabet.
3. *Well-formed formulæ* defined over this alphabet.

Let us look at the various components of this definition in the order in which they appear above.

Alphabet The alphabet is usually a set⁵ of symbols. In this case, we consider the set to be partitioned into two disjoint subsets: the set of *logical symbols* (common to all first-order languages) and the set of *non-logical* (or *extra-logical*) *symbols* (which are specific to a domain of interest). For example, all first-order languages will use a (logical) symbol to denote conjunction. If we are considering orderings on a set, we will probably also have the $<$ symbol as one of the non-logical symbols.

The set of *logical symbols* contains the following elements:

- The logical connectives $\wedge$ (conjunction), $\vee$ (disjunction), $\neg$ (negation), $\rightarrow$ (implication) and $\leftrightarrow$ (logical equivalence).
- The propositional constants *true* and *false*.

⁵All the sets we consider below are finite or denumerable.

- The quantifiers $\exists$ (exists) and $\forall$ (forall).
- Some punctuation symbols such as brackets “(” and “)” and comma “,”.
- An (denumerably) infinite set V of *variables*, written $X, Y, Z, \dots$

The *non-logical* (or *extra-logical*) symbols are defined by a *signature with predicates* (Σ, Π) . This is a pair in which the first element, Σ is the *function signature*, that is a set of function symbols, each considered with its own arity.⁶ The second element of the pair, Π , is the *predicate signature*, a set of predicate symbols together with their arities. Functions of arity 0 are said to be *constants* and are denoted by the letters $a, b, c, \dots$. Function symbols of positive arity are, as usual, denoted by $f, g, h, \dots$, while predicate symbols are denoted by $p, q, r, \dots$. Let us assume that the sets Σ and Π have an empty intersection and are also disjoint from the other sets of symbols listed above. The difference between function and predicate symbols is that the former must be interpreted as functions, while the latter must be interpreted as relations. This distinction will become clearer when we discuss formulæ.

Terms The concept of term, which is fundamental to mathematical logic and Computer Science, is used implicitly in many contexts. For example, an arithmetic expression is a term obtained by applying (arithmetic) operators to operands. Other types of construct, too, such as strings, binary trees, lists, and so on, can be conveniently seen as terms which are obtained using appropriate constructors.

In the simplest case, a term is obtained by applying a function symbol to as many variables and constants as required by its arity. For example, if a and b are constants, X and Y are both variables and f and g have arity 2, then $f(a, b)$ and $g(a, X)$ are terms. Nothing prevents the use of terms as the arguments to a function, provided that the arity is always respected. We can, for example, write $g(f(a, b), Y)$ or $g(f(a, f(X, Y)), X)$ and so on.

In the most general possible case, we can define terms as follows.

Definition 12.1 (Terms) The terms over a signature Σ (and over the set, V , of variables) are defined inductively⁷ as follows:

- A variable (in V) is a term.
- If f (in Σ) is a function symbol of arity n and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term.

As a particular case of the second point, a constant is a term. According to the letter of the definition, a term which corresponds to a constant must be written with parentheses: $a(), b(), \dots$. Let us establish, for ease of reading, that in the case of function symbols of arity 0, parentheses are omitted. Terms without variables are said to be *ground* terms. Terms are usually denoted by the letters $s, t, u, \dots$. Note that predicates do not appear in terms; they appear in formulæ (to express the properties of terms).

⁶Recall that, as stated above, the arity denotes the number of arguments of a function or relation.

⁷See the box on page 153 for inductive definitions.

Formulæ The *well-formed formulæ* (or *formulæ* for short) of the language allow us to express the properties of terms which, from the semantic viewpoint, are properties of some particular domain of interest. For example, if we have the predicate $>$, interpreting it in the usual fashion, writing $>(3, 2)$, we want to express the fact that the term “3” corresponds to a value which is greater than that associated with the term “2.” Predicates can then be used to construct complex expressions using logical symbols. For example, the formula $>(X, Y) \wedge >(Y, Z) \rightarrow >(X, Z)$ expresses the transitivity of $>$; it asserts that if $>(X, Y)$ is true and $>(Y, Z)$ is also true then, it is the case that $>(X, Z)$.

Wanting to define formulæ precisely, we have first atomic formulæ (or atoms), constructed by the application of a predicate to the number of terms required by its arity. For example, if p has arity 2, using two of the terms introduced above, we can write $p(f(a, b), f(a, X))$. Using logical connectives and quantification, we can construct complex formulæ from atomic ones. As usual, we have an inductive definition (or, equivalently, a free grammar—see Exercise 1).

Definition 12.2 (Formulæ) The (well-formed) formulæ over the signature with terms (Σ, Π) are defined as follows:

1. If $t_1, \dots, t_n$ are terms over the signature Σ and $p \in \Pi$ is a predicate symbol of arity n , then $p(t_1, \dots, t_n)$ is a formula.
2. *true* and *false* are formulæ.
3. If F and G are formulæ, then $\neg F$, $(F \wedge G)$, $(F \vee G)$, $(F \rightarrow G)$ and $(F \leftrightarrow G)$ are formulæ.
4. If F is a formula and X is a variable, then $\forall X.F$ and $\exists X.F$ are formulæ.

12.2.2 Logic Programs

A formula of first-order logic can have a highly complex structure which often determines the effort required to find a proof. In automatic theorem proving, and also in logic programming, particular classes of formulæ, called *clauses*, have been identified which lend themselves to more efficient manipulation, in particular using a special inference rule called *resolution*. A restricted version of the concept of clause, called *definite clauses*, as well as restricted forms of resolution (*SLD resolution*—see page 392) are of interest to us. Using them, the procedure for seeking a proof is not only particularly simple but also allows the explicit calculation of the values of the variables necessary for the proof. These values can be considered as the result of the computation, giving way to an interesting model of computation based on logical deduction. We will see this model in more detail below, for now we will concentrate on syntactic aspects.

Definition 12.3 (Logic Program) Let $H, A_1, \dots, A_n$ be atomic formulæ. A definite clause (for us simply a “clause”) is a formula of the form:

$$H : -A_1, \dots, A_n.$$

If $n = 0$, the clause is said to be a *unit*, or a fact, and the symbol $:-$ is omitted (but not the final full stop). A logic program is a set of clauses, while a pure PROLOG program is a sequence of clauses. A query (or goal) is a sequence of atoms $A_1, \dots, A_n$.

Let us clarify some points about this definition. First, the symbol $:-$ which we did not include in our alphabet, is simply a symbol denoting (reversed) implication ($\leftarrow$) and is the same as often encountered in real logic languages.⁸

The commas in a clause or in a query, from the logical viewpoint, should be interpreted as logical conjunction. The notation " $H :- A_1, \dots, A_n$." is therefore an abbreviation for " $H \leftarrow A_1 \wedge \dots \wedge A_n$." Note that the full stop is part of the notation and is important because it tells a potential interpreter or compiler that the clause it is working on has terminated.

The part on the left of $:-$ is called the *head* of the clause, while that on the right is called the *body*. A fact is therefore a clause with an empty body. A program is a set of clauses in the case of the theoretical formalism. In the case of PROLOG, on the other hand, a program is considered as a sequence because, as we will see, the order of clauses is relevant. Here, we used the simplified terminology found in many recent texts (for example, [1]). For more precise terminology, see the next box. The set of clauses containing the predicate symbol p in their head is said to be the *definition* of p . Variables occurring in the body of a clause and not in the head are said to be *local* variables.

12.3 Theory of Unification

The fundamental computational mechanism in logic programming is the solution of equations between terms using the unification procedure. In this procedure, substitutions are computed so that variables and terms can be bound (or instantiated). The composition of the different substitutions obtained in the course of a computation provides the result of the calculation. Before seeing the computational model for logic programming in detail, we must analyse unification in a little detail, a task we undertake in this section.

12.3.1 The Logic Variable

Before going into detail on the process of unification, it is important to clarify that the concept of variable we are considering is different from that seen in Sect. 6.2.1. Here, we consider the so-called *logic variable* which, analogously to the variables

⁸: $:-$ is used for pragmatic reasons. When logic languages were being introduced, it was much easier to type $:-$ rather than a left arrow.